Intern Assignment: Distributed Job Scheduler

Final Submission Document

---

1. System Architecture

The system follows a classic decoupled worker-queue model utilizing a monolithic backend API and standalone worker nodes, operating against a shared relational database.

```
graph TD
    subgraph Frontend
        UI[React.js Web Dashboard]
    end

    subgraph API Layer
        API[Django REST API]
    end

    subgraph Data Layer
        DB[(PostgreSQL/SQLite)]
    end

    subgraph Compute Layer
        W1[Worker Node 1]
        W2[Worker Node 2]
        W3[Worker Node N]
    end

    UI -- HTTP/REST Polling --> API
    API -- Read/Write Jobs --> DB
    W1 -- Polling + select_for_update --> DB
    W2 -- Polling + select_for_update --> DB
    W3 -- Polling + select_for_update --> DB
    W1 -- Heartbeat --> DB
```

---

2. Entity-Relationship (ER) Diagram

The database schema is heavily normalized to support multi-tenancy, granular queue control, lifecycle execution logs, and scheduled jobs.

```
erDiagram
    ORGANIZATION ||--o{ PROJECT : contains
    PROJECT ||--o{ QUEUE : manages
    QUEUE ||--o{ JOB : contains
    QUEUE ||--o{ SCHEDULED_JOB : schedules
    QUEUE }|--|| RETRY_POLICY : has_default
    JOB ||--o{ JOB_EXECUTION : creates
    JOB }|--|| RETRY_POLICY : has
    JOB ||--|{ DEAD_LETTER_QUEUE : sends_to
    JOB_EXECUTION ||--o{ JOB_LOG : generates
    WORKER ||--|{ WORKER_HEARTBEAT : sends
    WORKER ||--o{ JOB_EXECUTION : performs

    ORGANIZATION {
        int id PK
        string name
        int owner_id FK
    }
    PROJECT {
        int id PK
        string name
        int org_id FK
    }
    QUEUE {
        int id PK
        string name
        int project_id FK
        int priority
        boolean is_paused
    }
    JOB {
        uuid id PK
        int queue_id FK
        string state
        datetime execute_after
        int retries_count
    }
    JOB_EXECUTION {
        int id PK
        uuid job_id FK
        string status
        uuid worker_id FK
    }
    WORKER {
        uuid id PK
        string hostname
        boolean is_active
    }
```

---

3. Design Decisions & Trade-offs

Concurrency and Locking Strategy

Decision: We utilized Database Row-Level Locking (`SELECT ... FOR UPDATE SKIP LOCKED`) instead of an in-memory datastore like Redis or a message broker like RabbitMQ.
Trade-off:

- *Pros:* Radically simplifies infrastructure requirements since no external broker is needed. The database becomes the single source of truth, ensuring absolute ACID compliance for job states.
- *Cons:* Polling a relational database creates higher I/O overhead at extreme scale compared to Redis. However, by using index optimization and `SKIP LOCKED`, we prevent lock contention and enable safe distributed polling for thousands of concurrent workers.

API Polling vs WebSockets

Decision: The React dashboard uses short-polling (`setInterval`) every 2 seconds rather than WebSockets.
Trade-off:

- *Pros:* Easier to implement, completely stateless, and avoids the complexity of configuring Daphne/ASGI servers and managing WebSocket connection drops.
- *Cons:* Slight latency in real-time updates and higher HTTP overhead.

Retry Strategies

Decision: Implemented dynamic calculation for linear and exponential backoff dynamically at failure time, updating an `execute_after` timestamp.
Trade-off: Workers simply poll for jobs where `execute_after <= now()`, unifying immediate jobs and delayed retries into a single query path.

---

4. API Documentation

`GET /api/queues/`

Retrieves a list of all queues, their state, and active statistics.
Response:

```
{
  "queues": [
    {
      "id": 1,
      "name": "default",
      "priority": 1,
      "concurrency_limit": 10,
      "is_paused": false
    }
  ]
}
```

POST /api/jobs/

Dispatches a new background job into a specified queue.
Body:

```
{
  "name": "Process Video",
  "queue_id": 1,
  "payload": {"video_url": "https://..."}
}
```

POST /api/queues/<id>/toggle/

Pauses or Resumes a specific queue. When paused, workers will automatically skip claiming jobs from this queue.

POST /api/jobs/<id>/retry/

Manually forces a failed job back into the queue for re-execution, bypassing standard retry policy delays.

---

5. Setup & Execution Instructions

Ensure you have Python 3.x and Node.js installed.

1. **Install Dependencies:**

```
pip install django
npm install
```

2. **Database Migrations:**

```
python manage.py makemigrations core
python manage.py migrate
```

3. **Start the System:**
   Open three separate terminals and run:

- `python manage.py runserver` (Starts API)
- `python manage.py runworker` (Starts Worker Node)
- `npm start` (Starts React UI)

4. **Run Automated Tests:**

```
python manage.py test
```